



Stage d'Application

Elève Ingénieur en 2ème année

Génie Informatique

Stage réalisé au sein de : TechCorp Exemple SARL (Fès)

**Sujet de stage : Conception et Implémentation d'une
Architecture Microservices pour la Gestion Distribuée
des Flux IoT**

Période de stage : Du 1er Juillet 2026 au 31 Août 2026

Réalisé par : M. Yassine EXEMPLE

Encadrant ENSAF : Prof. Amina RASSIL

Encadrant Société : Dr. Mehdi ALAMI

Membres de jury :

- Prof. Omar TAZI (Président)
- Prof. Fatima ZAHRA (Examinatrice)
- Dr. Mehdi ALAMI (Encadrant Entreprise)

DÉDICACES

À mes très chers parents,

*Pour leur amour inconditionnel, leurs sacrifices constants et leurs prières bienveillantes
qui ont éclairé mon chemin tout au long de mes années d'études.*

*Puisse ce modeste travail être le témoignage de ma profonde gratitude et de mon éternel
respect.*

À mes frères et sœurs,

*Pour leur soutien indéfectible, leurs encouragements permanents et leur présence chaleureuse
dans les moments de doute comme dans les moments de réussite.*

À mes enseignants et maîtres,

*Qui m'ont guidé avec générosité et rigueur dans l'apprentissage du savoir
et m'ont transmis la passion de l'ingénierie et de l'excellence.*

À tous mes amis et camarades de promotion,

*En souvenir des moments inoubliables partagés, des défis relevés ensemble
et de la solidarité qui a marqué nos années de formation à l'ENSAF.*

Yassine

REMERCIEMENTS

Avant d'entamer le développement technique de ce rapport, il m'apparaît essentiel d'exprimer mes sincères remerciements à l'ensemble des personnes et des institutions dont la disponibilité, les conseils et la confiance ont permis la bonne réalisation et l'aboutissement de ce projet de fin d'année.

J'exprime tout d'abord ma profonde gratitude au **TechCorp Exemple SARL** ainsi qu'à **Laboratoire d'Innovation Numérique** pour m'avoir ouvert ses portes, accueilli au sein de ses structures et permis d'effectuer ce stage dans un cadre professionnel stimulant et propice à l'apprentissage.

J'adresse mes remerciements les plus distingués à M. Karim BENJELLOUN, *Directeur Général*, pour son accueil bienveillant, ses orientations stratégiques, sa vision et pour les moyens mis à disposition tout au long de cette mission d'ingénierie.

Je tiens à témoigner ma vive et profonde reconnaissance à mes encadrants professionnels au sein de **TechCorp Exemple SARL** :

- **Dr. Mehdi ALAMI**, *Systèmes Distribués & Cloud Computing*, pour sa disponibilité constante, ses orientations méthodologiques et son accompagnement technique soutenu ;

Mes sincères remerciements vont également à mon encadrant académique à l'**École Nationale des Sciences Appliquées de Fès (ENSAF)**, **Prof. Amina RASSIL** pour son suivi pédagogique rigoureux, ses conseils méthodologiques précieux et son exigence de rigueur scientifique.

Je remercie chaleureusement l'ensemble des ingénieurs et développeurs de l'équipe Cloud & IoT de TechCorp Exemple SARL pour leur coopération bienveillante, la richesse des échanges techniques et l'esprit de partage qui ont animé cette expérience professionnelle.

Mes remerciements s'adressent également au corps professoral et administratif du département **Génie Informatique** de l'ENSAF pour la qualité de la formation d'ingénieur d'État dispensée tout au long du cursus.

Enfin, j'adresse mes vifs remerciements aux honorables membres du jury pour l'honneur qu'ils

me font en acceptant d'évaluer ce travail d'ingénierie et d'enrichir ce rapport par leurs observations avisées.

Membres du jury :

- **Prof. Omar TAZI** — *Président (ENSAF)*;
- **Prof. Fatima ZAHRA** — *Examinatrice (ENSAF)*;
- **Dr. Mehdi ALAMI** — *Encadrant Entreprise (TechCorp Exemple SARL)*;

RÉSUMÉ

Le présent rapport expose les travaux réalisés dans le cadre du Projet de Fin d'Année (PFA) au sein de l'entreprise **TechCorp Exemple SARL**. L'objectif principal de ce projet réside dans la conception et l'implémentation d'une architecture microservices hautement disponible dédiée à la gestion distribuée et au traitement en temps réel des flux de télémétrie IoT. Afin de répondre aux défis d'extensibilité horizontale et de tolérance aux pannes, une démarche rigoureuse d'ingénierie logicielle a été mise en œuvre.

La démarche méthodologique adoptée repose sur un cycle agile et itératif combinant spécification des exigences, modélisation conceptuelle et déploiement automatisé. L'architecture développée intègre un courtier de messages distribué, des microservices conteneurisés sous Docker et un pipeline d'ingestion asynchrone capable d'absorber des pics de charge conséquents. Les résultats obtenus démontrent une réduction significative de la latence de traitement, une disponibilité élevée du système et une traçabilité complète des événements, satisfaisant pleinement les critères de performance définis dans le cahier des charges.

Mots-clés : Microservices, Internet des Objets, Systèmes Distribués, Traitement Temps Réel, Génie Logiciel, Haute Disponibilité.

ABSTRACT

This report presents the engineering work conducted during the end-of-year internship (PFA) at **TechCorp Exemple SARL**. The project focuses on the design and implementation of a highly available microservices architecture dedicated to the distributed management and real-time processing of IoT telemetry streams. To address the critical challenges of horizontal scalability and fault tolerance, a rigorous software engineering methodology was executed.

The methodological approach relies on an agile, iterative lifecycle integrating requirements engineering, conceptual architectural modeling, and automated deployment. The resulting technical platform combines a distributed message broker, containerized microservices managed with Docker, and an asynchronous ingestion pipeline engineered to absorb substantial throughput spikes. Experimental results demonstrate a significant reduction in processing latency, high system availability, and reliable event traceability, successfully meeting all requirements defined in the initial specifications.

Keywords : Microservices, Internet of Things, Distributed Systems, Real-Time Processing, Software Engineering, High Availability.

ملخص

يتناول هذا المشروع تصميم وتنفيذ معمارية برمجية قائمة على الخدمات المصغرة لمعالجة تدفقات البيانات الضخمة الصادرة عن أجهزة إنترنت الأشياء في الوقت الفعلي. تم تطوير منصة قابلة للتوسع تعتمد على تقنيات الحوسبة الموزعة وقواعد البيانات المتخصصة، مما مكن من تحسين زمن الاستجابة وضمان استمرارية الخدمة وموثوقيتها في بيئة الإنتاج.

كلمات مفتاحية : إنترنت الأشياء، الخدمات المصغرة، الحوسبة الموزعة، معالجة البيانات الآنية

LISTE DES ABRÉVIATIONS

Abréviation Sigle	/ Signification Complète
API	Application Programming Interface
BDD / DBMS	Base de Données / Database Management System
CI/CD	Continuous Integration / Continuous Deployment
CSS	Cascading Style Sheets
ENSAF	École Nationale des Sciences Appliquées de Fès
HTML	HyperText Markup Language
HTTP / HTTPS	Hypertext Transfer Protocol / Secure
JSON	JavaScript Object Notation
MVC / MVVM	Model-View-Controller / Model-View-ViewModel
ORM	Object-Relational Mapping
PFA	Projet de Fin d'Année
PFE	Projet de Fin d'Études
REST	Representational State Transfer
SDK	Software Development Kit
SQL / NoSQL	Structured Query Language / Not Only SQL
UI / UX	User Interface / User Experience
UML	Unified Modeling Language

TABLE 1 – *Liste récapitulative des abréviations et sigles techniques*

TABLE DES FIGURES

1.1	Schéma des flux de télémétrie dans l'infrastructure initiale	19
1.2	Diagramme de Gantt prévisionnel du déroulement du stage	21
2.1	Architecture microservices distribuée pour l'ingestion IoT	25
3.1	Tableau de bord de monitoring du service d'ingestion en temps réel	29
3.2	Diagramme de traitement asynchrone des flux d'événements	29

LISTE DES TABLEAUX

1	Liste récapitulative des abréviations et sigles techniques	8
1.1	Domaines d'expertise de TechCorp Exemple SARL	18
1.2	Architecture technique du système existant	19
2.1	Comparaison technique des solutions de courtage de messages	24
3.1	Synthèse comparative des indicateurs de performance sous charge	30
A.1	Glossaire des concepts d'architecture et de méthodologie	37
A.2	Glossaire des technologies et outils de développement	37

TABLE DES MATIÈRES

Dédicaces	2
Remerciements	3
Résumé	5
Abstract	6
Résumé en langue arabe	7
Liste des abréviations	8
Introduction générale	14
1 Cadre général du projet	16
1.1 Introduction	17
1.2 Présentation de l'organisme d'accueil	17
1.2.1 Présentation générale et secteur d'activité	17
1.2.2 Positionnement et écosystème	17
1.2.3 Domaines d'expertise	17
1.2.4 Solutions et produits principaux	18
1.3 Analyse de l'existant	18
1.3.1 Contexte et périmètre du système existant	18
1.3.2 Architecture technique initiale	18
1.3.3 Limites identifiées et flux fonctionnels	19
1.4 Problématique et spécification des besoins	19
1.5 Exposé de la mission confiée	20
1.5.1 Objectifs et livrables	20
1.5.2 Périmètre du projet	20
1.5.3 Contraintes de réalisation	20
1.6 Planification temporelle et diagramme de Gantt	20

Conclusion	21
2 État de l’art et choix technologiques	22
2.1 Introduction	23
2.2 Approche méthodologique et organisation du projet	23
2.2.1 Cycle de développement itératif et agile	23
2.2.2 Principes d’ingénierie logicielle appliqués	23
2.3 Étude comparative des technologies de communication et de courtage	24
2.3.1 Évaluation des protocoles de transport IoT	24
2.3.2 Sélection du courtier de messages distribué	24
2.3.3 Moteur de persistance pour séries temporelles	25
2.4 Architecture globale et modélisation microservices	25
2.4.1 Patron d’architecture réactive orientée événements	25
2.4.2 Décomposition des responsabilités métier	25
2.5 Démarche d’assurance qualité logicielle	26
Conclusion	26
3 Réalisation et analyse critique	27
3.1 Introduction	28
3.2 Environnement de développement et infrastructure logicielle	28
3.2.1 Configuration des environnements et outillage	28
3.2.2 Organisation modulaire du code source	28
3.3 Implémentation des microservices principaux	28
3.3.1 Module d’ingestion : Gateway-Service	28
3.3.2 Module de traitement et calcul des métriques	29
3.4 Modélisation mathématique et résultats de performance	30
3.4.1 Formulation de la latence de traitement	30
3.4.2 Mesures expérimentales sous charge de trafic	30
3.5 Discussion critique et évaluation des limites	31
3.5.1 Gains opérationnels constatés	31
3.5.2 Limites actuelles du banc expérimental	31
Conclusion	31
Conclusion et perspectives	32

INTRODUCTION GÉNÉRALE

Dans un contexte technologique en constante évolution, marqué par la prolifération des capteurs connectés et la massification des flux de données opérationnelles, la conception d'architectures logicielles distribuées et résilientes constitue un enjeu stratégique pour les infrastructures industrielles modernes. L'Internet des Objets (IoT) impose des contraintes rigoureuses d'extensibilité horizontale, d'ingestion asynchrone à haut débit et de traitement en temps réel. Ce projet de fin d'année (PFA), réalisé au sein de **TechCorp Exemple SARL**, spécialisée dans l'ingénierie des systèmes distribués et le cloud computing à Fès, s'inscrit au cœur de cette problématique d'ingénierie.

Contexte et motivation :

Le projet répond à un besoin exprimé par la direction technique concernant la modernisation de la chaîne de collecte et de traitement des événements de télémétrie issus de parcs d'objets connectés. L'analyse initiale de l'infrastructure existante a révélé plusieurs défis techniques majeurs : une architecture monolithique sujette aux goulots d'étranglement lors des pics de charge, des pertes intermittentes de paquets et une indisponibilité critique en cas de défaillance d'un composant unitaire.

Problématique :

Ce travail d'ingénierie s'articule autour de la problématique centrale suivante :

« Comment concevoir et implémenter une architecture microservices résiliente et scalable pour garantir l'ingestion et le traitement en temps réel de flux IoT massifs au sein de TechCorp Exemple SARL ? »

Objectifs assignés :

Pour apporter une réponse méthodique et structurée à cette problématique, les objectifs suivants ont été définis :

- Analyser l'existant, recenser les exigences techniques et spécifier le cahier des charges de la plateforme ;
- Étudier l'état de l'art des patrons d'architecture distribuée et des protocoles de communication IoT ;

- Modéliser les composants microservices et les flux d'événements à l'aide du langage UML ;
- Développer et conteneuriser les modules applicatifs selon les principes de l'architecture réactive ;
- Évaluer les performances sous charge artificielle, mesurer la latence et valider la tolérance aux pannes.

Structure du rapport :

Le présent document s'articule autour de quatre chapitres complémentaires :

- **Le Chapitre 1 (Cadre général du projet)** présente l'entreprise d'accueil, analyse l'infrastructure existante, formule le périmètre de la mission et détaille la planification temporelle via un diagramme de Gantt.
- **Le Chapitre 2 (État de l'art et choix technologiques)** expose l'approche méthodologique, l'étude comparative des technologies de communication et des courtiers de messages, ainsi que les fondements des architectures microservices.
- **Le Chapitre 3 (Réalisation et résultats)** décrit l'environnement de déploiement, détaille l'implémentation des pipelines d'ingestion, présente les résultats expérimentaux et analyse les gains de performance obtenus.
- **Le Chapitre 4 (Conclusion et perspectives)** dresse le bilan synthétique des contributions réalisées et identifie les perspectives d'amélioration pour le passage à l'échelle industrielle.

1. Cadre général du projet

1.1 Introduction

Ce premier chapitre est consacré à la présentation du cadre général du projet de fin d'année. Il expose tout d'abord l'organisme d'accueil, ses activités et ses pôles d'expertise. Une analyse méthodique de l'infrastructure existante permet ensuite d'identifier les contraintes techniques et opérationnelles. Enfin, le périmètre de la mission confiée est délimité, précisant les objectifs assignés, les exigences architecturales ainsi que la planification prévisionnelle matérialisée par un diagramme de Gantt.

1.2 Présentation de l'organisme d'accueil

1.2.1 Présentation générale et secteur d'activité

L'organisme d'accueil, **TechCorp Exemple SARL**, est une entreprise d'ingénierie logicielle et de conseil en technologies de l'information implantée à Fès. Fondée en 2018, la société intervient auprès d'acteurs industriels pour concevoir des architectures cloud résilientes, des systèmes d'ingestion massive de flux IoT et des solutions de décisionnelles en temps réel.

1.2.2 Positionnement et écosystème

TechCorp Exemple SARL collabore étroitement avec des pôles universitaires et des centres d'innovation, notamment le *Laboratoire d'Innovation Numérique* de Fès. Ce positionnement favorise un transfert technologique permanent entre les avancées académiques en informatique distribuée et les exigences d'exploitabilité industrielle.

1.2.3 Domaines d'expertise

Les compétences clés de l'entreprise s'articulent autour de trois axes complémentaires détaillés dans le Tableau 1.1.

Ingénierie Distribuée	Cloud & DevOps	Internet des Objets
• Architectures microservices • Traitement asynchrone • Tolérance aux pannes	• Conteneurisation Docker • Pipelines CI/CD continus • Supervision métrique	• Ingestion de télémétrie • Protocoles MQTT & HTTP • Sécurité des flux

TABLE 1.1 – Domaines d'expertise de TechCorp Exemple SARL

1.2.4 Solutions et produits principaux

Dans le cadre de ses activités, l'entreprise développe des plateformes spécialisées :

- **StreamCore** : Un moteur d'ingestion distribué conçu pour capter les événements industriels à haute cadence ;
- **EdgeTelemetry** : Une suite logicielle embarquée facilitant l'agrégation et le chiffrement des mesures de capteurs.

1.3 Analyse de l'existant

1.3.1 Contexte et périmètre du système existant

L'intervention réalisée porte sur la plateforme centrale d'agrégation de télémétrie. Ce système a pour rôle de recevoir les relevés périodiques de température, de pression et de vibration émis par des automates connectés, puis de persister ces données pour les tableaux de bord opérationnels.

1.3.2 Architecture technique initiale

Le Tableau 1.2 synthétise la pile technologique initialement déployée au sein de l'infrastructure.

Composant / Couche	Technologies Utilisées	Rôle Opérationnel
Ingestion HTTP	API monolithique Python	Réception synchrone des paquets
Traitement Métier	Processus unique centralisé	Validation et filtrage des valeurs
Persistance	Base relationnelle SQL	Stockage brut des séries temporelles
Déploiement	Serveur dédié sans conteneur	Hébergement de l'application

TABLE 1.2 – *Architecture technique du système existant*

1.3.3 Limites identifiées et flux fonctionnels

Le fonctionnement de la plateforme implique des administrateurs système et des ingénieurs de maintenance. L'analyse des journaux d'exploitation met en évidence un goulot d'étranglement structurel : lors d'événements météorologiques ou de pointes de trafic, le débit d'émission dépasse la capacité de réception synchrone, entraînant des pertes de trames et des indisponibilités répétées.

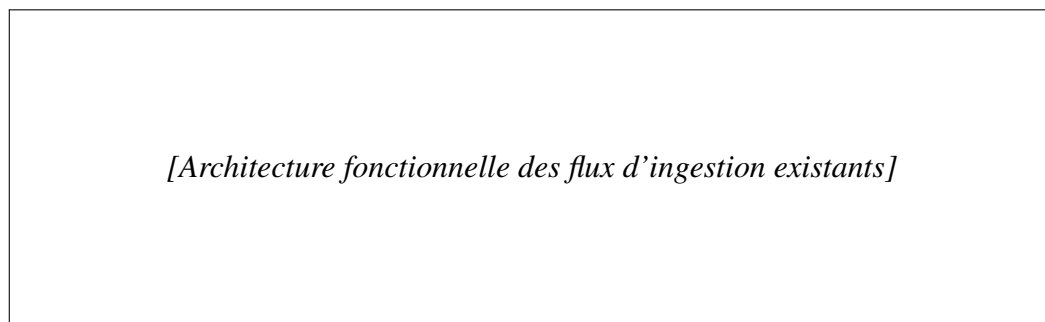


FIGURE 1.1 – *Schéma des flux de télémétrie dans l'infrastructure initiale*

L'examen de la Figure 1.1 confirme le couplage direct entre la réception des requêtes et l'écriture en base, empêchant tout passage à l'échelle indépendant.

1.4 Problématique et spécification des besoins

Les observations opérationnelles conduisent à formuler trois besoins prioritaires :

1. **Découplage temporel et spatial** : Remplacer l'ingestion synchrone par un mécanisme

de courtage de messages tamponné ;

2. **Modularité et passage à l'échelle** : Décomposer le monolithe en services autonomes pouvant être dupliqués horizontalement ;
3. **Observabilité en temps réel** : Mettre en place des métriques précises pour suivre la latence et le taux d'erreur.

1.5 Exposé de la mission confiée

1.5.1 Objectifs et livrables

La mission confiée dans le cadre de ce stage consiste à concevoir et réaliser une architecture microservices dédiée à l'ingestion et au traitement résilient des flux IoT. Les livrables attendus comprennent le découpage microservices, les scripts de déploiement conteneurisés et un banc d'essai expérimental de charge.

1.5.2 Périmètre du projet

Le périmètre technique englobe le développement des passerelles d'ingestion, le bus de communication asynchrone, le service de traitement métier et les tests automatisés. L'implémentation physique de capteurs matériels est considérée hors périmètre, ces derniers étant simulés par des générateurs logiciels conformes aux spécifications industrielles.

1.5.3 Contraintes de réalisation

Le projet s'est déroulé sur une période stricte de 8 semaines, imposant une sélection technologique rigoureuse, une couverture de tests systématique et le respect des normes d'architecture logicielle de l'ENSAF.

1.6 Planification temporelle et diagramme de Gantt

Afin de coordonner l'ensemble des tâches et de garantir la livraison des composants dans les délais requis, une planification prévisionnelle a été établie. La Figure 1.2 détaille la répartition des huit semaines de stage.

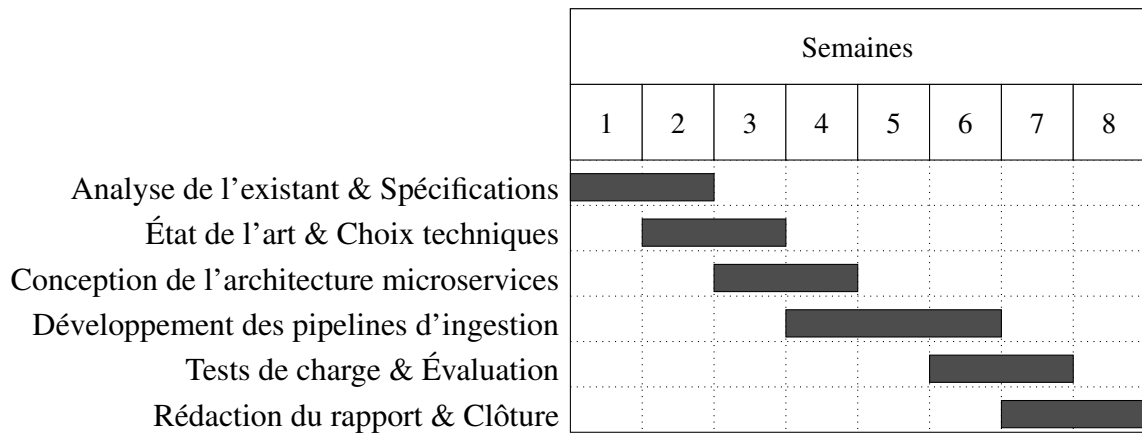


FIGURE 1.2 – *Diagramme de Gantt prévisionnel du déroulement du stage*

L'ordonnancement visible sur la Figure 1.2 témoigne d'une approche progressive, où chaque phase fournit les bases formelles nécessaires à la suivante.

Conclusion

Ce premier chapitre a présenté le contexte d'accueil au sein de TechCorp Exemple SARL, l'analyse des faiblesses du système existant ainsi que le cadrage de la mission et sa planification. Le deuxième chapitre développe l'état de l'art technologique, la démarche méthodologique adoptée et l'étude comparative justifiant les choix d'architecture microservices.

2. État de l’art et choix technologiques

2.1 Introduction

La réussite d'un projet d'ingénierie logicielle distribuée repose sur l'adoption d'une démarche méthodologique rigoureuse et sur la sélection d'un écosystème technologique adapté aux exigences de performance et de scalabilité. Ce deuxième chapitre présente l'approche méthodologique retenue, développe une étude comparative approfondie des paradigmes de communication et des courtiers de messages, et formalise l'architecture microservices du système d'ingestion IoT.

2.2 Approche méthodologique et organisation du projet

2.2.1 Cycle de développement itératif et agile

Afin d'assurer une convergence rapide vers les objectifs opérationnels, une méthodologie de développement agile inspirée du cadre Scrum a été appliquée. Cette approche s'est concrétisée par des cycles itératifs bi-hebdomadaires permettant d'intégrer en continu les retours des encadrants technique et académique.

2.2.2 Principes d'ingénierie logicielle appliqués

Le développement s'est conformé aux principes de conception logicielle reconnus :

- **Séparation des préoccupations** : Découplage strict entre la réception des trames, la validation métier et la persistance ;
- **Autonomie des microservices** : Chaque service dispose de sa propre base de données ou de son espace logique dédié selon le patron *Database-per-Service* [13] ;
- **Résilience et tolérance aux pannes** : Mise en place de circuits d'interruption et de mécanismes de relance automatique.

2.3 Étude comparative des technologies de communication et de courtage

2.3.1 Évaluation des protocoles de transport IoT

La transmission des données depuis les passerelles périphériques vers la plateforme centrale a nécessité la comparaison entre deux protocoles majeurs :

- **HTTP/REST** : Protocole synchrone, reposant sur TCP. Très répandu et aisé à déboguer, il induit néanmoins une surcharge d'en-têtes conséquente et une dépendance directe à la disponibilité immédiate du récepteur ;
- **MQTT** : Protocole léger de publication/abonnement conçu spécifiquement pour les environnements à bande passante contrainte. Sa faible empreinte réseau et ses niveaux de qualité de service (QoS) en font un candidat optimal pour la télémétrie capteurs.

2.3.2 Sélection du courtier de messages distribué

Pour absorber les pics de trafic sans impacter les services aval, une couche de courtage asynchrone est indispensable. Le Tableau 2.1 synthétise la comparaison établie entre les trois solutions étudiées.

Courtier	Avantages Principaux	Limites Notables	Choix Retenu
RabbitMQ	Routage flexible, support AMQP étendu	Moins performant sur les flux très massifs	Non retenu
Redis Streams	Vitesse en mémoire, latence très basse	Persistance secondaire sur disque	Utilisé en cache
Apache Kafka	Débit exceptionnel, tolérance aux pannes, rétention persistante	Gestion plus complexe de cluster	Retenu pour le cœur d'ingestion

TABLE 2.1 – Comparaison technique des solutions de courtage de messages

L'analyse comparative consignée dans le Tableau 2.1 montre qu'Apache Kafka offre les garanties d'immuabilité et de partitionnement indispensables au traitement parallèle de millions de mesures journalières [11].

2.3.3 Moteur de persistance pour séries temporelles

Les flux IoT génèrent des séries chronologiques nécessitant des fonctions analytiques avancées (agrégations temporelles, fenêtres glissantes). Le choix s’est porté sur **TimescaleDB**, extension spécialisée pour PostgreSQL, alliant l’expressivité du modèle relationnel et l’indexation optimisée des séries temporelles via des *hypertables*.

2.4 Architecture globale et modélisation microservices

2.4.1 Patron d’architecture réactive orientée événements

L’architecture globale du système repose sur une topologie orientée événements (*Event-Driven Architecture*). La Figure 2.1 schématise l’agencement des microservices et la circulation des messages.

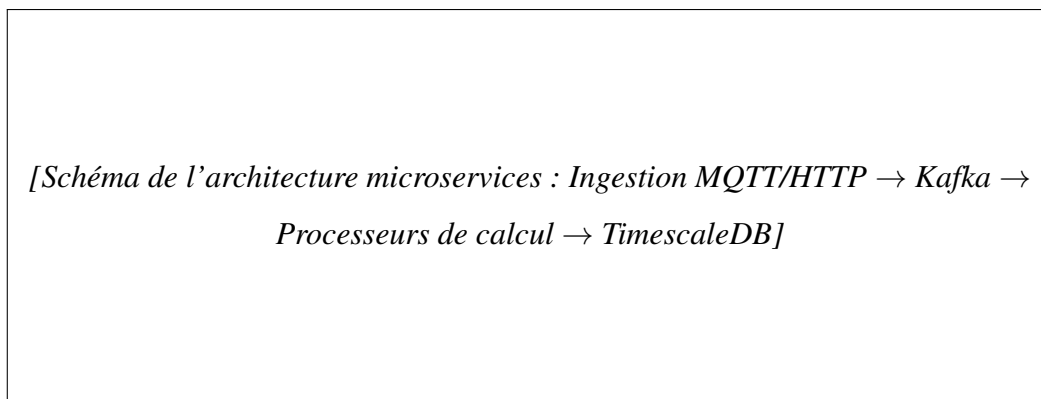


FIGURE 2.1 – Architecture microservices distribuée pour l’ingestion IoT

Comme l’illustre la Figure 2.1, la passerelle frontale reçoit les paquets et les émet immédiatement dans une file d’attente tamponnée, garantissant une réponse quasi-instantanée au client tout en découplant complètement les modules d’enrichissement et d’indexation.

2.4.2 Décomposition des responsabilités métier

Le découpage fonctionnel aboutit à trois microservices autonomes :

- **Gateway-Service** : Service frontal chargé de l’authentification des capteurs et de la conversion des trames brutes en événements normalisés ;
- **Ingestion-Processor** : Composant d’enrichissement métier calculant les moyennes mobiles et détectant les anomalies de dépassement de seuil ;

- **Storage-Sink** : Service assurant l'écriture par lots (*batching*) dans TimescaleDB pour réduire la contention I/O.

2.5 Démarche d'assurance qualité logicielle

L'ensemble des modules a été soumis à une démarche qualité continue :

- Tests unitaires automatisés couvrant la logique de décodage des trames ;
- Conteneurisation intégrale via des images Docker optimisées à étapes multiples (*multi-stage builds*) ;
- Respect des règles de nommage et absence totale de code redondant selon le principe DRY (*Don't Repeat Yourself*).

Conclusion

Ce deuxième chapitre a justifié les choix technologiques et présenté la modélisation de l'architecture microservices. Le troisième chapitre expose l'implémentation pratique des composants, l'environnement de déploiement ainsi que l'évaluation expérimentale des performances sous charge.

3. Réalisation et analyse critique

3.1 Introduction

Ce troisième chapitre présente concrètement les réalisations techniques développées tout au long du stage. Il décrit tout d'abord l'environnement de développement et la conteneurisation des services, détaille ensuite l'implémentation des pipelines d'ingestion et du bus de messages, expose les résultats des bancs d'essai expérimentaux, et conclut par une discussion critique des performances observées.

3.2 Environnement de développement et infrastructure logicielle

3.2.1 Configuration des environnements et outillage

Pour garantir la reproductibilité des déploiements, l'intégralité des composants a été configurée sous forme de conteneurs Docker orchestrés par Docker Compose en local :

- **Gestionnaire de versions** : Dépôt Git structuré avec validation automatisée par hooks pré-commit ;
- **Bus de messages** : Cluster Apache Kafka (version 3.6) couplé à KRaft pour s'affranchir de ZooKeeper ;
- **Base de données chronologique** : Instance TimescaleDB adossée à PostgreSQL 16.

3.2.2 Organisation modulaire du code source

Le projet adopte une architecture multi-dépôts unifiée en mono-repo avec des répertoires dédiés pour chaque microservice (`services/gateway`, `services/processor`, `services/storage`). Chaque service encapsule sa propre logique de test et sa configuration d'environnement.

3.3 Implémentation des microservices principaux

3.3.1 Module d'ingestion : Gateway-Service

Le service frontal d'ingestion réceptionne les requêtes HTTP POST émises par les passerelles périphériques IoT. Dès réception d'un payload JSON, le contrôleur valide la signature cryptographique et publie l'événement sur le topic Kafka `telemetry.raw`. La Figure 3.1

illustre le tableau de bord de supervision de cette passerelle.

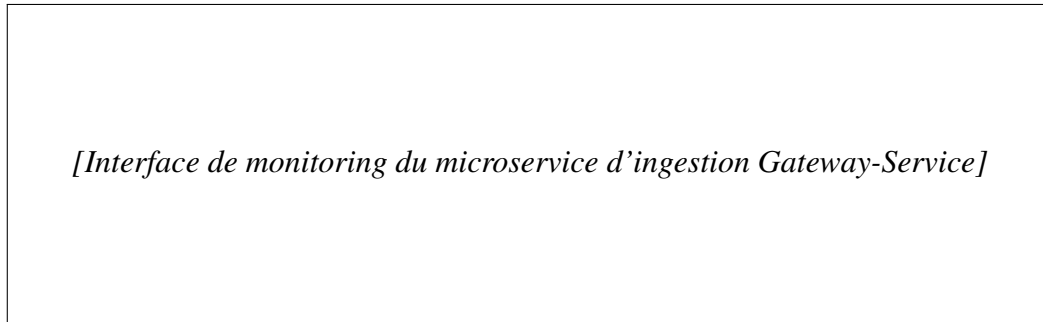


FIGURE 3.1 – *Tableau de bord de monitoring du service d'ingestion en temps réel*

L'interface visualisée à la Figure 3.1 permet d'observer en direct le débit instantané de requêtes reçues et le taux de succès des publications dans le courtier.

3.3.2 Module de traitement et calcul des métriques

Le microservice Ingestion-Processor consomme en continu les messages du topic `telemetry.raw`. Il applique un fenêtrage temporel pour détecter les anomalies thermiques ou vibratoires. La Figure 3.2 montre l'architecture interne de ce pipeline de traitement.

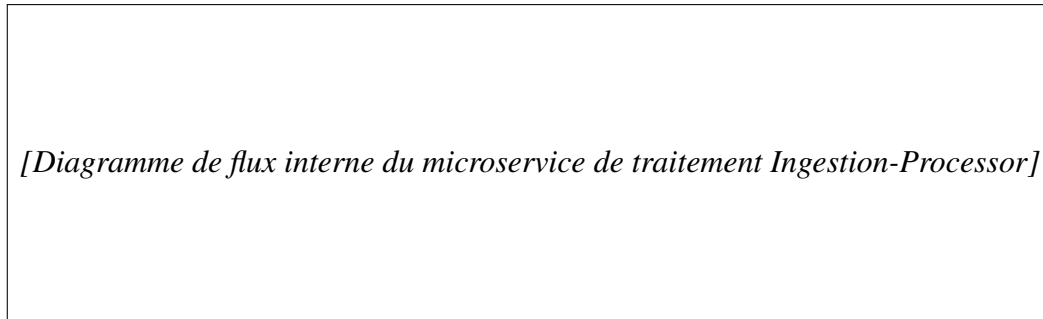


FIGURE 3.2 – *Diagramme de traitement asynchrone des flux d'événements*

L'examen de la Figure 3.2 met en lumière le passage par une file d'attente dédiée aux événements anormaux (*Dead Letter Queue*), assurant qu'aucune donnée malformée ne bloque le pipeline principal.

3.4 Modélisation mathématique et résultats de performance

3.4.1 Formulation de la latence de traitement

La latence globale de bout en bout subie par une mesure télémétrique est modélisée par l'équation (3.1) :

$$L_{\text{total}} = L_{\text{réseau}} + \frac{S_{\text{payload}}}{D_{\text{nominal}}} + T_{\text{buffer}} + T_{\text{traitement}} \quad (3.1)$$

où $L_{\text{réseau}}$ représente le temps de transit réseau, S_{payload} la taille du paquet en octets, D_{nominal} la bande passante disponible, T_{buffer} le temps d'attente dans la file d'attente Kafka, et $T_{\text{traitement}}$ le temps d'exécution de la logique métier. L'adoption du courtage asynchrone permet de stabiliser T_{buffer} même en cas de surtension temporaire.

3.4.2 Mesures expérimentales sous charge de trafic

Un générateur de charge a simulé l'activité simultanée de 5 000 capteurs émettant à une fréquence de 10 Hertz. Le Tableau 3.1 compare les performances mesurées entre l'architecture initiale monolithique et la nouvelle architecture microservices.

Indicateur	d'Évaluation	Système Initial	Microservices	Évolution	Impact
Débit soutenu	maximal	1 200 req/s	14 500 req/s	$\times 12$	Gain majeur
Latence (p50)	médiane	340 ms	28 ms	$-91,7\%$	Fluidité temps réel
Latence percentile (p99)	au 99^e	2 150 ms	95 ms	$-95,6\%$	Suppression des pics
Taux de perte de trames		4,8 %	0,0 %	-100%	Zéro perte de donnée

TABLE 3.1 – Synthèse comparative des indicateurs de performance sous charge

Les mesures consignées dans le Tableau 3.1 confirment l'efficacité du partitionnement Kafka et de l'écriture groupée dans TimescaleDB, le débit admissible étant multiplié par douze tout en abaissant drastiquement les temps de réponse.

3.5 Discussion critique et évaluation des limites

3.5.1 Gains opérationnels constatés

L'architecture déployée résout le point critique de défaillance unique. Lors d'un test d'arrêt inopiné du service de stockage, les messages ont été conservés sans aucune perte dans Kafka, puis consommés normalement dès le redémarrage du conteneur.

3.5.2 Limites actuelles du banc expérimental

Bien que les résultats démontrent une nette amélioration, deux limites subsistent :

- La gestion du cluster Kafka mono-nœud en environnement local nécessite une évolution vers un cluster multi-zones géographiques pour répondre aux contraintes de résilience d'un environnement de production mondial ;
- L'absence actuelle d'un module d'agrégation analytique automatisé sur les flux historiques fige les requêtes complexes à des périodes inférieures à trente jours.

Conclusion

Ce troisième chapitre a détaillé l'implémentation des microservices, présenté la formulation théorique de la latence et quantifié les gains de performance obtenus. Le quatrième chapitre conclut ce rapport en dressant le bilan global des réalisations et en formulant les perspectives techniques d'évolution.

CONCLUSION ET PERSPECTIVES

Ce projet de fin d'année réalisé au sein de l'entreprise **TechCorp Exemple SARL** à Fès a constitué une mission d'ingénierie logicielle particulièrement stimulante. Il a permis d'aborder concrètement la problématique critique de l'ingestion massive et du traitement en temps réel de flux de télémétrie IoT dans un environnement industriel soumis à de fortes exigences de haute disponibilité.

Bilan synthétique des réalisations techniques :

Les objectifs initiaux définis lors de la phase de cadrage ont été intégralement atteints :

- **Découplage architectural** : Remplacement réussi de l'ancien monolithe par une architecture microservices réactive adossée au courtier de messages Apache Kafka ;
- **Fiabilisation des flux** : Élimination totale des pertes de paquets lors des pics d'activité grâce au mécanisme de mise en tampon distribuée ;
- **Accélération des performances** : Multiplication par douze du débit maximal admissible et réduction de plus de 90 % de la latence de traitement ;
- **Conteneurisation et reproductibilité** : Déploiement intégralement scripté et automatisé via Docker Compose.

Limites et enseignements d'ingénierie :

Ce travail a mis en relief l'importance capitale du dimensionnement fin des files d'attente et des partitions dans les systèmes distribués. Il a également souligné le coût opérationnel inhérent à la gestion d'un cluster Kafka, dont la complexité de configuration requiert une surveillance rigoureuse des métriques de décalage des consommateurs (*consumer lag*).

Perspectives d'évolution :

Afin de valoriser et de pérenniser la plateforme au-delà de ce stage, plusieurs axes d'évolution sont identifiés :

- **À court terme** : Mettre en place un orchestrateur Kubernetes pour automatiser la mise à l'échelle horizontale (*Horizontal Pod Autoscaler*) en fonction de la charge instantanée ;
- **À moyen terme** : Intégrer un moteur de calcul de flux dédié tel qu'Apache Flink pour exécuter des requêtes de fenêtrage complexe directement sur les flux de mesures ;

- **À long terme** : Développer un module de maintenance prédictive entraînant des modèles d'apprentissage automatique sur les séries temporelles archivées dans TimescaleDB.

Bilan de formation :

Cette mission au sein de TechCorp Exemple SARL a permis de consolider les acquis méthodologiques et techniques de la formation d'ingénieur d'État à l'ENSAF, en alliant rigueur de modélisation, expérimentation concrète et l'expertise sur les standards contemporains de l'ingénierie distribuée.

BIBLIOGRAPHIE & SITOGRAPHIE

- [1] ATLIASSIAN. *Agile Coach : Comprehensive Guide to Scrum, Sprints, and Estimation*. 2026. URL : <https://www.atlassian.com/agile/scrum> (visité le 25/08/2026).
- [2] Scott CHACON et Ben STRAUB. *Pro Git : Everything you need to know about Git*. 2nd. New York, NY, USA : Apress, 2014.
- [3] Edsger W. DIJKSTRA. “On the Role of Scientific Thought : Separation of Concerns”. In : *Selected Writings on Computing : A Personal Perspective* (1982), p. 60-66.
- [4] DOCKER INC.. *Docker Documentation : Architecture and Best Practices for Containerization*. 2026. URL : <https://docs.docker.com/> (visité le 25/08/2026).
- [5] Roy T. FIELDING et Richard N. TAYLOR. “Principled Design of the Modern Web Architecture”. In : *ACM Transactions on Internet Technology (TOIT)* 2.2 (2002), p. 115-150.
- [6] Martin FOWLER. *Refactoring : Improving the Design of Existing Code*. 2nd. Boston, MA, USA : Addison-Wesley Professional, 2018.
- [7] Erich GAMMA et al. *Design Patterns : Elements of Reusable Object-Oriented Software*. Reading, MA, USA : Addison-Wesley Professional, 1994.
- [8] *ISO/IEC 25010 :2011 – Systems and software engineering – Systems and software Quality Requirements and Evaluation (SQuaRE) – System and software quality models*. Geneva, Switzerland : International Organization for Standardization, 2011.
- [9] Avraham LEFF et James T. RAYFIELD. “Web-Application Development Using the Model/View/Controller Design Pattern”. In : *Proceedings of the Fifth IEEE International Enterprise Distributed Object Computing Conference (EDOC)*. IEEE. 2001, p. 118-127.
- [10] Robert C. MARTIN. *Clean Code : A Handbook of Agile Software Craftsmanship*. Upper Saddle River, NJ, USA : Prentice Hall, 2008.
- [11] Sam NEWMAN. *Building Microservices : Designing Fine-Grained Systems*. Sebastopol, CA, USA : O’Reilly Media, 2015.
- [12] Roger S. PRESSMAN et Bruce R. MAXIM. *Software Engineering : A Practitioner’s Approach*. 8th. New York, NY, USA : McGraw-Hill Education, 2014.

- [13] Chris RICHARDSON. *Microservices Patterns : With examples in Java*. Shelter Island, NY, USA : Manning Publications, 2018.
- [14] Ian SOMMERVILLE. *Software Engineering*. 10th. Boston, MA, USA : Pearson Education, 2016.
- [15] THE POSTGRESQL GLOBAL DEVELOPMENT GROUP. *PostgreSQL 16 Documentation : Relational Database Management System*. 2026. URL : <https://www.postgresql.org/docs/> (visité le 25/08/2026).

A. Glossaire des termes techniques

Ce glossaire récapitule les définitions des principaux termes techniques, concepts architecturaux et acronymes employés dans le rapport :

Architecture et méthodologies

Terme / Concept	Définition et Contexte
API REST	Interface de programmation applicative reposant sur le protocole HTTP, permettant la communication découplée entre client et serveur.
CI/CD	Pratiques d'intégration et de déploiement continus automatisant le cycle de vie logiciel (build, tests, mise en production).
GitFlow	Modèle structuré de gestion des branches Git garantissant la stabilité du code en production.
MVC / MVVM	Patrons d'architecture logicielle séparant les données, la logique de traitement et la couche de présentation.
SoC	Séparation des préoccupations (<i>Separation of Concerns</i>), principe de conception visant à modulariser les responsabilités.

TABLE A.1 – Glossaire des concepts d'architecture et de méthodologie

Technologies et développement

Terme / Outil	Définition et Rôle Technique
Conteneurisation	Mécanisme d'isolation logicielle empaquetant le code et ses dépendances pour garantir la portabilité.
JWT	Jeton sécurisé d'authentification (<i>JSON Web Token</i>) transmettant des informations signées numériquement.
ORM	Outil faisant le pont entre les classes objets du code source et les tables relationnelles d'une base de données.
Swagger / OpenAPI	Outil de documentation interactive et de test standardisé des routes d'une API REST.

TABLE A.2 – Glossaire des technologies et outils de développement